Chapter 3

Learning and Evaluation

Xu Kuang, Stanford GSB

We have seen in the earlier chapter how to use stochastic modeling (the Erlang M/M/c/k systems) and the resulting Erlang formulas to guide decisions around how many processing resources to allocate for a given level of throughput and probability of delay. The discussion there, however, was largely predicated on us accepting the model as already determined and readily available, and also that it correctly captures reality.

Recall from Chapter **??** the Frame-Learn-Use-Evaluate modeling workflow. In this chapter, we will discuss the procedure for learning a specific congestion model within the broader family and the questions that arise concerning the eventual effectiveness of this learned model in guiding actions. The procedure for learning such a model is typically neither very difficult nor complex, but evaluating whether we have ended up with a good enough model can be more subtle and delicate.

3.1 The Basics of Learning the Congestion Model

In the case of congestion modeling in the Erlang style, we implicitly, via human expertise, frame the problem by adopting a mechanistic stochastic model that depicts the congestion dynamics arising from serving randomly arriving requests using the family of M/M/c/k queueing models. Each individual model within this family is further specified by several parameters. By learning a congestion model, we mean identifying the most appropriate combination of these parameters. They include:

1. The arrival rate λ: how many jobs arrive in the system per unit of time?
2. The service rate μ: how many jobs can each server or worker process per unit of time?
3. The number of servers or workers c.
4. The total number of jobs the system can contain, k. Alternatively, the size of the waiting buffer, $k - c$.

Out of these parameters, c and k are usually not considered something to be learned because they can be chosen by the designer themselves. This leaves determining the values of λ and μ to be learned from data.

Cold- vs. Warm-starts

In the case of a call center or data center, λ and μ are typically estimated using operational data from existing operations. The emphasis here is that we are assuming that *some* form of the system the model aims to capture has already been deployed in practice for long enough to collect such data. We call this the *warm-start* modeling problem, where the system is already in operation, affording modelers like Erlang some readily available data to get started. In contrast, there are equally many applications, if not more prevalent, in which the decision-maker faces a *cold-start* problem, where we hope to use modeling

to guide the design of a system that has yet to be launched or has only recently been launched.

The detailed statistical procedures employed to estimate and learn these parameters from data can vary greatly depending on the preference of the designer (Bayesian, Frequentist, etc.). As a baseline for our example, things are really quite simple. By choosing such a relatively simple queueing model, we are effectively baking in the assumption that the arrival and service rates are time-invariant, and in particular, that they do not depend on the day of week or time of day. With this restriction in place, and suppose that we are learning the parameter for a model of a call center, learning λ simply amounts to:

1. Define a sufficiently long data collection horizon, T. Say we let this be one year, so $T = 525,600$ minutes.

2. Measure the number of incoming calls during this year, N. Suppose this number came to be $N = 28,598,331$ calls.

3. Then λ is estimated to be $N/T = 54.41$ calls per minute.

The estimation of μ can be accomplished in a similar manner, by taking the average of processing times of jobs over a sufficiently long time horizon.

To be more precise, the estimation procedure above corresponds to the frequentist maximum-likelihood estimator for λ underz the time-invariance assumption. Some practitioners may also prefer a Bayesian procedure, where they can incorporate stronger prior beliefs about what λ should be. In this example, a large number of observations (across a whole year) is collected to estimate a relatively large arrival rate, and as such these two approaches are unlikely to yield different conclusions. But in cases where arrivals occur sparingly, such as when estimating dynamics related to highly specialized equipment in the METRIC model discussed in Chapter **??**, the practitioner's prior could play a much more significant role.

3.2 Do Erlang Queues Capture Reality?

Being able to estimate the parameters in a congestion model is, however, only half the battle. A common mode of failure of stochastic modeling in the real world stems from choosing the wrong model family to begin with. That is, implicit assumptions deep within the constitution of the model can make it so fundamentally deviant from reality from the outset that even sound statistical procedures for estimating the various parameters cannot make up for these more fundamental departures.

In the Erlang examples, there are many such possible assumptions that could, in theory, prevent the model from reliably portraying reality. To name just a few, the inter-arrival times may not be exponentially distributed (a fact called out by Erlang himself in the original 1909 paper). The arrival rate is very likely not stationary over time in almost any real-world application. Servers may not work at the same speed. And finally, customers may not patiently wait in line to receive service indefinitely...

Some of these assumptions may be necessary evils to cope with in exchange for a relatively elegant and robust mathematical model that is still useful enough, while others may be so egregious as to become deal-breakers. Without further evaluation, a practitioner simply cannot tell one from another.

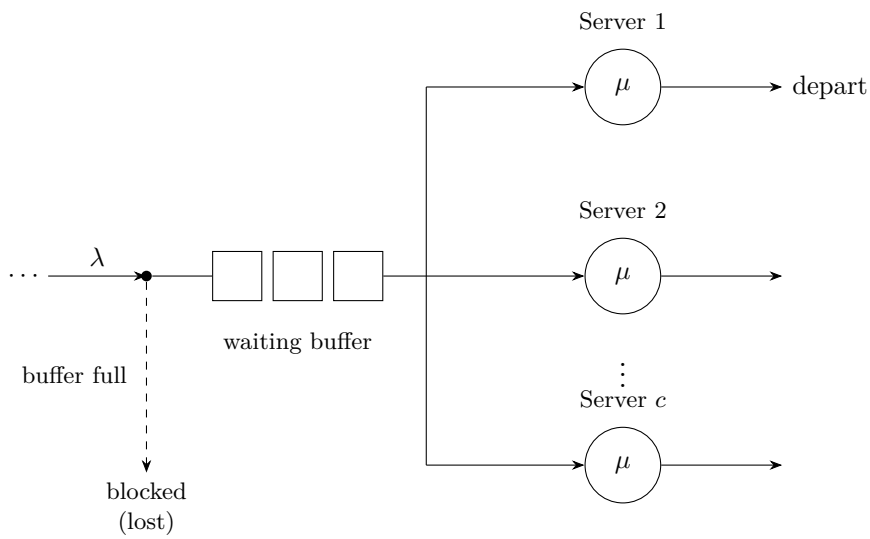

Figure 3.1: Illustration of Erlang's $M/M/c/K$ queues (from Chapter **??**, reproduced here). Calls arrive at rate λ. If the buffer has room, the call waits until a server is free; each of the c servers has service rate μ. If the buffer is full (system at capacity $K = c +$ buffer size), the arriving call is blocked and lost.

3.2.1 Case Study: Statistical Analysis of a Call Center

For Erlang models specifically, (Brown et al., 2005) serves as an excellent case study of model evaluation using real-world data from an Israeli call center in 1999. We will examine some of the key findings there.

The basic setup is as follows: Brown et al. (2005) study a small Israeli bank call center using a full year (1999) of call-by-call operational data. The dataset includes over 1.2 million incoming calls, of which roughly 750,000 terminate in the VRU (voice response unit, the automated phone menu) and about 450,000 request a live agent. For calls that proceed past the VRU, the records include detailed time stamps for arrival, queue entry and exit, service start and end, and whether the caller is served or abandons, along with call-type labels (regular service, stock trading, internet support, and new customers). The center operates primarily on weekdays (Sunday–Thursday) from 7 AM to midnight, making the data rich enough to separately study arrivals, service durations, waiting, and abandonment behavior.

3.3 Recommended Reading

For primary empirical evidence on whether Erlang-style assumptions hold in a modern service system, see (Brown et al., 2005). For original historical context behind the Erlang family itself, see Chapter **??** and (Erlang, 1917).

Bibliography

Brown, L., Gans, N., Mandelbaum, A., Sakov, A., Shen, H., Zeltyn, S., and Zhao, L. (2005). Statistical analysis of a telephone call center: A queueing-science perspective. *Journal of the American Statistical Association*, 100(469):36–50.

Erlang, A. K. (1917). Solution of some problems in the theory of probabilities of significance in automatic telephone exchanges. *Elektroteknikeren*, 13:5–13. Original Danish title: Łosning af nogle Problemer fra Sandsynlighedsregningen af Betydning for de automatiske Telefoncentraler. Archived scan: Wayback copy.